

Ejercicio 6: Ciclos Hamiltonianos con Aristas Prohibidas

Enunciado

Considera un grafo completo no dirigido con N vértices $\{1, \dots, N\}$ y A aristas, donde hay $k < A$ aristas que no se pueden recorrer (prohibidas). Diseña un algoritmo para calcular el número de ciclos hamiltonianos en el grafo que no incluyen las aristas prohibidas.

Un grafo es completo si cada vértice está conectado a todos los demás vértices del grafo. Un ciclo hamiltoniano es un ciclo del grafo que visita cada vértice exactamente una vez. Los ciclos que incluyen las mismas aristas se cuentan solo una vez. Por ejemplo, los ciclos $\{1, 2, 3, 4, 1\}$, $\{1, 4, 3, 2, 1\}$ y $\{2, 3, 4, 1, 2\}$ son todos iguales (se cuentan una vez), mientras el ciclo $\{1, 3, 2, 4, 1\}$ es diferente de los anteriores.

1. Análisis del Problema y Reducción de Simetrías

En un grafo completo no dirigido de N vértices, el número total de ciclos hamiltonianos posibles es $\frac{(N-1)!}{2}$.

Para evitar contar el mismo ciclo físico varias veces en distintas representaciones (rotaciones o inversiones como $\{1, 2, 3, 4, 1\}$ y $\{1, 4, 3, 2, 1\}$), aplicamos dos reglas de **reducción de simetría**:

1. **Fijar el vértice de inicio:** Todo ciclo hamiltoniano válido debe visitar el vértice 1. Por lo tanto, podemos fijar el vértice 1 como el inicio y el final absoluto de todos los caminos explorados. Esto elimina las rotaciones redundantes (ej. $\{2, 3, 4, 1, 2\}$ se reduce a $\{1, 2, 3, 4, 1\}$).
2. **Fijar la dirección (Evitar caminos invertidos):** Para evitar contar un ciclo y su reverso (ej. $\{1, 2, 3, 4, 1\}$ y $\{1, 4, 3, 2, 1\}$), podemos imponer que el primer paso desde el vértice 1 vaya a un vértice vecino u menor que el último paso antes de cerrar el ciclo v (es decir, $u < v$). Sin embargo, en un algoritmo de vuelta atrás es aún más fácil: **simplemente dividimos el resultado final del conteo de caminos dirigidos entre 2**.

2. Diseño del Algoritmo (Vuelta Atrás / Backtracking)

Dado que un grafo completo tiene todas las aristas posibles, representamos las aristas prohibidas mediante una **matriz de adyacencia de prohibición** P de tamaño $N \times N$, donde:

- $P[u][v] = \text{Verdadero}$ si la arista entre u y v está prohibida.
- $P[u][v] = \text{Falso}$ en caso contrario (arista permitida).

Estrategia de Búsqueda:

1. Empezamos en el vértice 1. Marcamos el vértice 1 como visitado.

2. Exploramos recursivamente todos los vértices vecinos no visitados v .
3. Solo transitamos de un nodo actual u a un vecino v si la arista (u, v) **no está prohibida** ($P[u][v] == \text{Falso}$).
4. Cuando hayamos visitado exactamente los N vértices del grafo:
 - Comprobamos si podemos regresar al vértice inicial 1 desde el último nodo de la ruta sin usar una arista prohibida.
 - Si es posible, sumamos 1 a nuestro contador de caminos hamiltonianos válidos orientados.
5. Al finalizar toda la exploración, el número de ciclos hamiltonianos únicos será el contador total dividido por 2 (para eliminar el sentido de giro del ciclo).

3. Algoritmo y Pseudocódigo

Matriz de Prohibición

Para agilizar las consultas, cargamos las k aristas prohibidas en un conjunto rápido o en una matriz booleana:

```

Función CalcularCiclosHamiltonianos(N, Aristas_Prohibidas):
    // 1. Inicializar matriz de prohibición
    P[1..N][1..N] = Falso
    Para cada (u, v) en Aristas_Prohibidas:
        P[u][v] = Verdadero
        P[v][u] = Verdadero

    // Vector para controlar qué vértices ya están en el camino actual
    visitado[1..N] = Falso
    visitado[1] = Verdadero // Fijamos el inicio en el vértice 1

    total_caminos = ContarCaminos(1, 1, N, visitado, P)

    // Cada ciclo físico no dirigido se cuenta dos veces (en sentido horario y an
    Retornar total_caminos / 2
  
```

Función de Conteo Recursiva (ContarCaminos)

```

Función ContarCaminos(nodo_actual, cont_visitados, N, visitado, P):
    // Caso Base: Hemos visitado todos los vértices
    Si cont_visitados == N:
        // Debemos poder cerrar el ciclo volviendo al vértice 1 sin usar una aris
        Si P[nodo_actual][1] == Falso:
            Retornar 1
        Sino:
            Retornar 0

    caminos_validos = 0

    // Probamos todos los posibles siguientes nodos 'siguiente' de 1 a N
  
```

```

Para siguiente de 1 a N:
    Si visitado[siguiente] == Falso y P[nodo_actual][siguiente] == Falso:
        // 1. Decisión tentativa: marcar como visitado
        visitado[siguiente] = Verdadero

        // 2. Exploración recursiva
        caminos_validos = caminos_validos + ContarCaminos(siguiente, cont_vis

        // 3. Backtracking: deshacer decisión para probar otras ramas
        visitado[siguiente] = Falso

Retornar caminos_validos

```

4. Análisis de Complejidad

- **Complejidad Temporal:**

- En el peor de los casos (cuando $k = 0$, es decir, no hay aristas prohibidas), el algoritmo explora todas las permutaciones posibles de los vértices restantes, lo que toma un tiempo de $O((N - 1)!)$.
- Con aristas prohibidas ($k > 0$), el espacio de búsqueda se reduce considerablemente (poda temprana), ya que muchas ramas se descartan inmediatamente cuando se detecta una transición prohibida.

- **Complejidad Espacial:**

- $O(N^2)$ para la matriz booleana de prohibición P .
- $O(N)$ para el vector de visitados y la pila de recursión del sistema (cuya profundidad máxima es exactamente N , que es el número total de vértices).